Find total ways to reach the n’th stair with at-most m steps

Given a stair case, find total number of ways to reach the n’th stair from bottom of the stair when a person is only allowed to take at-most m steps at a time.

For example,

```
Input: n = 3, m = 2
Output: Total ways to reach the 3’rd stair with at-most 2 steps are
3

1 step  + 1 step  + 1 step
1 step  + 2 steps
2 steps + 1 step
```

Custom Search

Browse

Adobe Algorithm Amazon
BFS Binary Search Bit Hacks DFS

```
Input: n = 4, m = 3
Output: Total ways to reach the 4’th stair with at-most 3 steps are
7

1 step  + 1 step  + 1 step  + 1 steps
1 step  + 1 step  + 2 steps
1 step  + 2 steps + 1 step
1 step  + 3 steps
2 steps + 1 step  + 1 step
2 steps + 2 steps
3 steps + 1 step
```

In previous post, we have discussed how to get number of ways to reach the `n'th` stair from bottom of the stair when a person is allowed to take at-most 3 steps at a time. In this post, we will extend the solution for at-most m steps.

For a maximum of 3 steps at a time, we have came up with the recurrence relation `T(n)`:

```
T(n) = T(n-1) + T(n-2) + T(n-3) where n >= 0 and
T(0) = 1, T(1) = 1, and T(2) = 2
```

For at-most m steps, the recurrence relation `T(n)` can be written as following:

```
T(n) = T(n-1) + T(n-2) + ... T(n-m)
```

i.e. we can reach the `n'th` stair from either `(n-1)'th` stair, `(n-2)'th` stair, `(n-3)'th`. ... `(n-m)'th` stair. Here’s C/Java program that implements the above recurrence:

FIFO Google Greedy Hashing Intro JSON LCS LIFO Maze Memoized Microsoft Must Know Priority Queue Probability Recursive Searching Sliding Window Tabulation Tricky Trie

Subscribe to posts

Enter your email address to subscribe to new posts and receive notifications of new posts by email.

Email Address

Subscribe

C

```c
#include <stdio.h>

// Recursive function to find total ways to reach the n'th sta
// when a person is allowed to take at-most m steps at a time
int totalWays(int n, int m)
{
    // base case: invalid input
    if (n < 0)
        return 0;

    // base case: 1 way (with no steps)
    if (n == 0)
        return 1;

    int count = 0;
    for (int i = 1; i <= m; i++)
        count += totalWays(n - i, m);

    return count;
}

// main function
int main(void)
{
    int n = 4, m = 3;

    printf("Total ways to reach the %d'th stair with at-most %
            n, m, totalWays(n, m));

    return 0;
}
```

Download Run Code

Java

```
class Main
{
    // Recursive function to find total ways to reach the n'th
    // when a person is allowed to take at-most m steps at a t
    public static int totalWays(int n, int m)
    {
        // base case: invalid input
        if (n < 0) {
            return 0;
        }

        // base case: 1 way (with no steps)
        if (n == 0) {
            return 1;
        }

        int count = 0;
        for (int i = 1; i <= m; i++) {
            count += totalWays(n - i, m);
        }

        return count;
    }

    public static void main(String[] args)
    {
        int n = 4, m = 3;

        System.out.printf("Total ways to reach the %d'th stair
                                "%d steps are %d", n, m, total
    }
}
```

Download Run Code

Output:

```
Total ways to reach the 4’th stair with at-most 3 steps are 7
```

The time complexity of above solution is exponential since it computes solutions to the same sub-problems again and again i.e. the problem exhibits overlapping subproblems.

The problem has an optimal substructure since a solution to a problem can be derived using solution to its sub-problems. Since both properties of dynamic programming are satisfied, we can use dynamic programming to bring down the time complexity to `O(mn)`. This can be done in either top-down or bottom-up fashion:

1. Top-Down Approach

We can use memoization to solve this problem in top-down fashion. The idea is to store the results of function calls and return the cached result when the same inputs occur again.

C

```c
#include <stdio.h>

// Recursive DP function to find total ways to reach the n'th
// when a person is allowed to take at-most m steps at a time
int totalWays(int n, int m, int dp[])
{
    // base case: invalid input
    if (n < 0)
        return 0;

    // base case: 1 way (with no steps)
    if (n == 0)
        return 1;
```

```

    // if the sub-problem is not seen before
    if (dp[n] == 0)
    {
        for (int i = 1; i <= m; i++)
            dp[n] += totalWays(n - i, m, dp);
    }

    // return the sub-problem solution
    return dp[n];
}

// main function
int main(void)
{
    int n = 4, m = 3;

    // create an array of n+1 size for storing solution to the
    int dp[n+1];

    // initialize the array by 0's
    memset(dp, 0, sizeof(int) * (n+1));

    printf("Total ways to reach the %d'th stair with at-most %
          n, m, totalWays(n, m, dp));

    return 0;
}
```

Download Run Code

Java

```
class Main
{
    // Recursive DP function to find total ways to reach the n
    // when a person is allowed to take at-most m steps at a t
    public static int totalWays(int n, int m, int[] dp)
    {
        // base case: invalid input
```

```
        if (n < 0) {
            return 0;
        }

        // base case: 1 way (with no steps)
        if (n == 0) {
            return 1;
        }

        // if the sub-problem is not seen before
        if (dp[n] == 0) {
            for (int i = 1; i <= m; i++)
                dp[n] += totalWays(n - i, m, dp);
        }

        // return the sub-problem solution
        return dp[n];
    }

    public static void main(String[] args)
    {
        int n = 4, m = 3;

        // create an array of n+1 size for storing solution to
        int[] dp = new int[n+1];

        System.out.printf("Total ways to reach the %d'th stair
                            "%d steps are %d", n, m, totalWays
    }
}
```

Download Run Code

Output:

```
Total ways to reach the 4’th stair with at-most 3 steps are 7
```

2. Bottom-Up Approach

We can also use tabulation to solve this problem in bottom-up fashion. The idea is to construct a temporary array which stores the results of each sub-problem using already computed results of the smaller sub-problems. The algorithm can be implemented as follows in C and Java:

C

```c
#include <stdio.h>

// Recursive function to find total ways to reach the n'th sta
// when a person is allowed to take at-most m steps at a time
int totalWays(int n, int m)
{
    // create an array of n+1 size for storing solutions to th
    int lookup[n + 1];

    // base case: 1 way (with no steps)
    lookup[0] = 1;

    // 1 way to reach the 1st stair
    lookup[1] = 1;

    // 2 ways to reach the 2nd stair
    lookup[2] = 2;

    // Fill the lookup table in bottom-up manner
    for (int i = 3; i <= n; i++)
    {
        lookup[i] = 0;
        for (int j = 1; j <= m && (i - j) >= 0; j++)
            lookup[i] += lookup[i - j];
    }

    return lookup[n];
}

```

```
// main function
int main(void)
{
    int n = 4, m = 3;

    printf("Total ways to reach the %d'th stair with at-most %
        n, m, totalWays(n, m));

    return 0;
}
```

Download Run Code

Java

```
class Main
{
    // Recursive function to find total ways to reach the n'th
    // when a person is allowed to take at-most m steps at a t
    public static int totalWays(int n, int m)
    {
        // create an array of n+1 size for storing solutions t
        int[] lookup = new int[n + 1];

        // base case: 1 way (with no steps)
        lookup[0] = 1;

        // 1 way to reach the 1st stair
        lookup[1] = 1;

        // 2 ways to reach the 2nd stair
        lookup[2] = 2;

        // Fill the lookup table in bottom-up manner
        for (int i = 3; i <= n; i++)
        {
            lookup[i] = 0;
            for (int j = 1; j <= m && (i - j) >= 0; j++) {
                lookup[i] += lookup[i - j];
            }
```

```
        }

        return lookup[n];
    }

    public static void main(String[] args)
    {
        int n = 4, m = 3;

        System.out.printf("Total ways to reach the %d'th stair
                          "%d steps are %d", n, m, totalWays
    }
}
```

Download Run Code

Output:

```
Total ways to reach the 4’th stair with at-most 3 steps are 7
```

⭐⭐⭐⭐⭐ (**5** votes, average: **5.00** out of 5)

Thanks for reading.

Please use our online compiler to post code in comments. To contribute, get in touch with us.
Like us? Please spread the word and help us grow. Happy coding 🙂

Sharing is caring:

Tweet Share 20 Share More

Dynamic Programming Memoized, Recursive, Tabulation

← Reverse a Doubly Linked List

Convert a Ternary Tree to a Doubly Linked List →

Leave a Reply

b i link b-quote u ul ol li code spoiler

Start the discussion (Please use an online compiler to post code in comments)

Subscribe

Techie Delight © 2019 All Rights Reserved. Privacy Policy Contact us